

AIMS System - Detailed Class Design Documentation

1. Design Class: Product

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	int	null	Unique Identifier for the product
2	title	String	null	Name of the product
3	category	String	null	Product category (Book, CD, DVD, or LP record)
4	value	float	0.0	Base price of the product before VAT
5	price	float	0.0	Selling price of the product
6	quantity	int	0	Available quantity in stock
7	description	String	null	Product description
8	imageUrl	String	null	The URL for the image of the product item
9	barCode	String	null	Unique barcode for the product
10	warehouseEntryDate	Date	null	Date when the product entered the warehouse
11	dimensions	String	null	Physical dimensions of the product (e.g., 10x20x5 cm)
12	weight	float	null	Weight of the product in kilograms
13	warehouseProvince	String	null	province of warehouse storing the product
14	warehouseDistrict	String	null	district of warehouse storing the product
15	warehouseAddress	String	null	detailed address of warehouse storing the product

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	checkProductAvailability	boolean	Checks if the product has sufficient quantity available
2	updateProduct	vold	Updates product information in the system
3	checkEligibilityOfRushDelivery	boolean	Checks if product is eligible for rush delivery
4	createProduct	vold	Create a new product in the system
5	searchProduct	vold	Search for a specific product attributes

Parameter:

- checkProductAvailability(quantity: int): Boolean
 - **quantity:** An integer representing the number of products to check if the available inventory is sufficient.
 - **Return:** true if the available inventory is sufficient, otherwise false.
- updateProduct(productId: int): vold
 - **productId:** productId must exist
- searchProduct(searchCriteria: Map<String, Object>): vold
 - **searchCriteria:** A map containing search criteria (field names and values)

Exception:

- updateProduct(productId: int): vold
 - ProductNotFoundException: If no product with the specified productId is found.
 - ExceedUpdateTimesException: If update more than 30 products a day
 - ExceedUpdatePriceTimesPerDayException: If update price more than twice a day

Method:

How to use parameters/attributes:

- checkProductAvailability(quantity: int): Boolean
 - **Purpose:** This method checks whether the available inventory is sufficient for the requested quantity of a product.
 - **Usage:** If the available stock is at least equal to the requested quantity, the method returns true; otherwise, it returns false.
- updateProduct(productId: int): void
 - **Purpose:** This method updates the details of an existing product based on the provided information.
 - **Usage:** If the product exists, the method updates attributes such as name, price, quantity, etc
- createProduct(): void
 - **Purpose:** This method creates a new product with the provided information.
 - **Usage:** If all required attributes are provided and valid, the product is successfully created and added to the inventory.
- checkEligibilityOfRushDelivery(address: String): Boolean
 - **Purpose:** to check rush delivery support
 - **Usage:** compare 'address' parameter (from delivery information) and address of product (based on information provided by Product Manager)
- searchProduct(searchCriteria: Map<String, Object>): void
 - **Purpose:** Searches for products matching specific criteria.
 - **Usage:** Used to find products based on various attributes.
 - **Processing:**
 1. Constructs a database query based on the search criteria
 2. Executes the query
 3. Returns matching products

State:

- If stock > requested quantity → Available.
- If stock < requested quantity → Not Available.
- If product_Id is invalid → ProductNotFoundException is raised.

2. Design Class: Cart

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	cartItemsList	List<CartItem>	empty	List of CartItem in the cart
2	cartId	float	0.0	Unique Identifier for the cart

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	calculateTotalPrice	float	Calculates and returns the total price
2	empty	void	Empties the cart
3	checkEligibilityOfRushDeliveryOfChosenItems	boolean	Checks if all items are eligible for rush delivery
4	checkAvailabilityOfChosenItems	boolean	Checks if all items have sufficient inventory
5	isEmpty	boolean	Checks if the cart is empty

Parameter:

- checkEligibilityOfRushDeliveryOfChosenItems(province: String, district: String): boolean
 - **province:** The name of the province where the delivery is requested.
 - **district:** The name of the district within the province.
 - **Return:** true if all products are eligible for rush delivery, otherwise false.

Exception: None

Method:

How to use parameters/attributes:

- calculateTotalPrice(): float
 - **Purpose:** Computes the total price for the cart.
 - **Usage:** Iterates through all cart items, sums their prices, applies VAT, and returns the total price.
- empty(): void

- **Purpose:** Empties the cart by removing all items.
 - **Usage:** All items are cleared, and subtotal, VAT, and total price are reset to zero.
- **checkEligibilityOfRushDeliveryOfChosenItems(province: String, district: String): boolean**
 - **Purpose:** Determines if all products in the cart are eligible for rush delivery.
 - **Usage:** Checks each product's delivery eligibility and returns true if all qualify, otherwise false.
- **checkAvailabilityOfChosenItems(province: String, district: String): boolean**
 - **Purpose:** Checks if all items in the cart have sufficient stock.
 - **Usage:** Iterates through the chosen cart items and verifies stock availability. Returns true if all items are available, otherwise false.
- **isEmpty(): boolean**
 - **Purpose:** Checks if the cart is empty.
 - **Usage:** Returns true if there are no items in the cart, otherwise false.

State:

- If cartItemsList is empty → Empty
- If cartItemsList contains at least one item → Active
- If checkout() is called and payment is not completed → CheckedOut
- If payment is successful → Pald
- If cart is emptied (either manually or after order completion) → Empty

3. Class Design: CartItem

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	int	null	Id of the product in the cart
2	quantity	int	0	Quantity of the product
3	cartId	int	null	Unique Identifier for the cart
4	cartItemId	int	null	Unique Identifier for the cart item

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	changeQuantityOfProducts.	vold	Updates the quantity and recalculates subtotal
2	addProductToCart	vold	Adds a product to the cart with specified quantity

Parameter:

- addProductToCart(product: Product, quantity: int): volds
 - **product:** A Product object representing the product to add.
 - **quantity:** An integer representing the number of products to add.
 - **Return:** None.
- changeQuantityofProduct(product: Product, quantity: int): vold
 - **product:** A Product object representing the product to update.
 - **quantity:** An integer representing the new quantity.
 - **Return:** None.

Exception:

- **addProductToCart(productId: int, quantity: int): vold**
 - **ProductNotFoundException:** Raised if the specified product does not exist in the system.
 - **InsufficientStockException:** Raised if the requested quantity exceeds the available stock.
- **changeQuantityOfProduct(productId: int, quantity: int): vold**
 - **ProductNotFoundException:** Raised if the product is not found in the cart.
 - **InsufficientStockException:** Raised if the requested quantity exceeds the available stock.

Method:

How to use parameters/attributes:

- **addProductToCart(ProductId: int):**
 - **Purpose:** Adds a product to the cart with a specified quantity if stock is sufficient.
 - **Usage:** If the product exists and the requested quantity is available, it is added to the cart. Otherwise, InsufficientStockException is raised.

- **changeQuantityofProduct(ProductId: int):**
 - **Purpose:** Updates the quantity of an existing product in the cart.
 - **Usage:** If the product is already in the cart and the stock allows, the quantity is updated. Otherwise, InsufficientStockException is raised.

State

- **Available** :If stock > requested quantity, the product is available for purchase.
- **Not Available** :If stock < requested quantity, the requested quantity cannot be fulfilled.
- **Product Not Found**:If product_Id is invalld (i.e., the product does not exist in the system), a ProductNotFoundException is raised

4. Design Class: PlaceOrderController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	currentCart	Cart	null	Reference to the customer's current shopping cart
2	currentOrder	Order	null	Order being created in the current process
3	deliveryInfo	DeliveryInfo	null	Delivery information for the current order
4	currentInvoice	Invoice	null	Invoice generated for the current order
5	placeOrderService	PlaceOrderService	null	provide service for Controller

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	requestToPlaceOrder	boolean	Initiates the order placement process with items from cart
2	checkAvailabilityOfChosenItems	boolean	Verifies if all items in cart are available in sufficient quantity
3	Order	Order	Creates a new order with items from the cart
4	DeliveryForm	DeliveryForm	Creates and returns a delivery form for customer input
5	submitDeliveryInfo	boolean	Processes the delivery information submitted by customer
6	checkValldityOfDeliveryInfo	boolean	Valldates the delivery information format and completeness
7	DeliveryInfo	DeliveryInfo	Creates a delivery info object from valldated input
8	calculateShippingFee	float	Calculates shipping fee based on product weight and delivery location
9	Invoice	Invoice	Creates an invoice with order and delivery information
10	placeOrder	boolean	Finalizes the order placement after payment is processed
11	save	vold	Saves the confirmed order to the database
12	empty	vold	Clears the cart after successful order placement
13	sendSuccessOrderEmail	boolean	Sends order confirmation email to customer
14	OrderInfoScreen	OrderInfoScreen	Creates a screen displaying order and transaction details
15	placeRushOrder	boolean	Verify if there exists any products for rush delivery from cart

Parameter:

- **requestToPlaceOrder(Cart): boolean**
 - **cartObj:** The shopping cart containing products to be ordered

- **Return:** true if all products are available and order can proceed, false otherwise
- **checkAvailabilityOfChosenItems(Cart): boolean**
 - **cart:** The shopping cart to check availability for
 - **Return:** true if all items are available in sufficient quantity, false otherwise
- **submitDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
 - **deliveryInfo:** Map containing delivery information fields (name, address, etc.)
 - **Return:** true if delivery information is valld, false otherwise
- **checkValldityOfDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
 - **deliveryInfo:** Map containing delivery information to valldate
 - **Return:** true if all required fields are valld, false otherwise
- **calculateShippingFee(deliveryInfo: DeliveryInfo, order: Order): float**
 - **deliveryInfo:** Customer's delivery information including location
 - **order:** The order containing products with weight information
 - **Return:** The calculated shipping fee in the local currency
- **placeOrder(transactionInfo: Map<String, String>): boolean**
 - **transactionInfo:** Payment transaction details
 - **Return:** true if order was successfully placed, false otherwise
- **placeRushOrder(province: String, district: String): boolean**
 - **province:** entered by a Administrator
 - **district:** entered by a Administrator
 - **Return:** true if there exists any cart items for rush delivery

Exception:

- **requestToPlaceOrder(Cart): boolean**
 - **EmptyCartException:** Thrown if cart is empty
 - **InvaliddCartException:** Thrown if cart contains invalld items
- **checkAvailabilityOfChosenItems(Cart): boolean**
 - **DatabaseConnectionException:** Thrown if unable to check product availability
 - **InsufficientStockException:** Details which products have insufficient stock
- **submitDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
 - **InvaliddDeliveryInfoException:** Details which fields are invalld or missing
- **requestToPlaceOrder(Cart): boolean**
 - **UnsupportedLocationException:** Thrown if delivery location is not supported
 - **InvaliddWeightException:** Thrown if product weight data is missing or invalld

Method:

- **requestToPlaceOrder(cartObj: Cart): boolean**
 - **Purpose:** Initiates the order placement process by checking product availability
 - **Processing:**
 - Valldates that the cart is not empty
 - For each item in the cart:
 - Calls checkProductAvailability on the Product object
 - If any product is unavailable, returns false and provldes details on unavailable items
 - If all products are available, creates a new Order object
 - Creates and returns a DeliveryForm for customer input
 - **Return:** true if order process can continue, false if any products are unavailable
- **submitDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
 - **Purpose:** Processes and valldates delivery information provided by customer
 - **Processing:**
 - Valldates all required delivery information fields
 - Creates a DeliveryInfo object with the valldated information
 - Calculates shipping fee based on product weights and delivery location
 - Creates an Invoice with order details, products, VAT, and shipping fee
 - Returns an InvoiceScreen for customer review
 - **Return:** true if delivery information is valld and processing continues, false otherwise
- **placeOrder(transactionInfo: Map<String, String>): boolean**
 - **Purpose:** Finalizes order placement after successful payment
 - **Processing:**
 - Saves the order to the database with "Pending Processing" status
 - Empties the cart
 - Sends order confirmation email to customer
 - Creates and displays OrderInfoScreen with order and transaction details
 - **Return:** true if order was successfully placed, false if any step fails

State:

- Before requestToPlaceOrder is called, currentCart contains the customer's selected products
- After successful requestToPlaceOrder, currentOrder contains a new Order object
- After submitDeliveryInfo, deliveryInfo contains valldated delivery information and currentInvoice contains the generated invoice

- After successful placeOrder, the order is saved to the database, the cart is emptied, and confirmation email is sent

5. Design Class: DeliveryInfo

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	recipientName	String	null	Name of the recipient
2	email	String	null	Email address
3	phoneNumber	String	null	Phone number
4	province	String	null	Province/city
5	district	String	null	District
6	address	String	null	Delivery address
7	deliveryMethod	String	"Regular"	Delivery method (Regular/Rush)
8	deliveryTime	String	0.0	Requested time for rush delivery
9	isRushDeliveryEligible	boolean	false	Indicates if address is eligible for rush delivery
10	deliveryInstructions	String	null	Special delivery instructions

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	DeliveryInfo	vold	Constructor for creating delivery information
2	valldateDeliveryInfo	boolean	Valldates all required delivery information fields
3	isEligibleForRushDelivery	boolean	Calculates shipping fee based on product weight and location
4	calculateShippingFee	float	Calculates shipping fee based on product weight and location
5	calculateRushDeliveryFee	float	Calculates rush delivery fee for eligible products
6	getDeliveryInfo	Map<String, String>	Returns all delivery information as a map
7	updateDeliveryInfo	boolean	Updates the delivery information with new values
8	isInHanoiInnerCity	boolean	Checks if delivery is to inner Hanoi districts

Parameter:

- **DeliveryInfo(recipientName: String, email: String, phoneNumber: String, province: String, district: String, address: String, deliveryMethod: String)**
 - **recipientName:** Full name of the recipient
 - **email:** Contact email address
 - **phoneNumber:** Contact phone number
 - **province:** Delivery province/city
 - **district:** Delivery district
 - **address:** Detailed delivery address
 - **deliveryMethod:** Delivery method (Regular or Rush)
- **valldateDeliveryInfo(): boolean**
 - **Return:** true if all required fields are valld, false otherwise
- **isEligibleForRushDelivery(): boolean**
 - **Return:** true if the address is in an area that supports rush delivery, false otherwise
- **calculateShippingFee(weight: float): float**
 - **weight:** Total weight of the products in kilograms
 - **Return:** The calculated shipping fee in VND
- **calculateRushDeliveryFee(numItems: int): float**
 - **numItems:** Number of items for rush delivery
 - **Return:** The calculated rush delivery fee in VND
- **getDeliveryInfo(): Map<String, String>**
 - **Return:** Map containing all delivery information fields
- **updateDeliveryInfo(updatedInfo: Map<String, String>): boolean**
 - **updatedInfo:** Map containing updated delivery information fields

- **Return:** true if update was successful, false otherwise
- **isInHanoiInnerCity(): boolean**
 - **Return:** true if the delivery district is in inner Hanoi city, false otherwise

Exception:

- **validateDeliveryInfo(): boolean**
 - **InvalidEmailException:** Thrown if email format is invalid
 - **InvalidPhoneNumberException:** Thrown if phone number format is invalid
 - **MissingRequiredFieldException:** Thrown if any required field is missing
- **calculateShippingFee(weight: float): float**
 - **InvalidWeightException:** Thrown if weight is negative or zero
 - **UnsupportedLocationException:** Thrown if delivery location is not supported

Method:

- **validateDeliveryInfo(): boolean**
 - **Purpose:** Validates all required delivery information fields for completeness and format
 - **Processing:**
 1. Checks that recipient name is not empty
 2. Validates email format using regex pattern
 3. Validates phone number format using regex pattern
 4. Checks that province, district, and address are not empty
 5. If delivery method is "Rush", checks that the address is eligible for rush delivery
 - **Return:** true if all validations pass, false otherwise with appropriate error messages
- **isEligibleForRushDelivery(): boolean**
 - **Purpose:** Determines if the delivery address is eligible for rush delivery
 - **Processing:**
 1. Checks if province is "Hanoi"
 2. Checks if district is one of the inner city districts of Hanoi
 - **Return:** true if in eligible area, false otherwise
- **calculateShippingFee(weight: float): float**
 - **Purpose:** Calculates the shipping fee based on product weight and delivery location
 - **Processing:**
 1. Checks if delivery is to inner city Hanoi or Ho Chi Minh City
 - If yes, base fee is 22,000 VND for first 3kg
 - If no, base fee is 30,000 VND for first 0.5kg
 2. Adds 2,500 VND for every additional 0.5kg
 3. Applies free shipping if order value exceeds 100,000 VND (up to 25,000 VND)
 - **Return:** The calculated shipping fee
- **calculateRushDeliveryFee(numItems: int): float**
 - **Purpose:** Calculates additional fee for rush delivery items
 - **Processing:**
 1. Multiplies number of rush delivery items by 10,000 VND
 2. Adds this to the base shipping fee
 - **Return:** The calculated rush delivery fee

State:

- After initialization, all attributes are set to their default values or to the values provided in the constructor
- After validation, the object represents a validated delivery information record
- If rush delivery is requested, deliveryTime and deliveryInstructions are set, and isRushDeliveryEligible flag is checked

6. Design Class: DeliveryForm

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	checkoutButton	javafx.scene.control.Button	null	button to save entered user information
2	labels	List<javafx.scene.control.Label>	null	“Username”, “Password”, ...
3	textFields	List<javafx.scene.control.TextField>	null	input components
4	placeOrderController	PlaceOrderController	null	Controller managing rush delivery condition

Table 2 . Operation design

#	Name	Return	Description
1	DeliveryForm	constructor	Constructor which initializes resource to render and handle DeliveryForm modal
2	addAdditionalDeliveryInformation	vold	display more field when Customer select rush delivery as delivery method

Parameter:

- DeliveryForm(user: User, placeOrderController: PlaceOrderController)
- addAdditionalDeliveryInformation(): void

Exception: no specific information

Method:

- UserInformationForm(user: User, userController: UserController)
 - **Purpose:** initialize a form to edit on.
 - **Usage:** called when you click edit/ create button on ‘DashboardView’ component.
- addAdditionalDeliveryInformation(): void
 - **Purpose:** display more field when Customer select rush delivery as delivery method
 - **Usage:** called when Customer selects rush delivery and there exists some items for rush delivery.

State: no specific information

7. Design Class: PlaceOrderService

Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

Table 2. Operation Design

#	Name	Return type	Description (purpose)
1	checkRushDeliverySupport	boolean	check rush delivery condition

Parameter:

- checkRushDeliveryCondition(province: String, district: String, cart: Cart): boolean
 - **province, district:** the provided address information
 - **cart:** containing list of CartItem.
 - **Return:** true if payment initiation or completion is successful, otherwise false

Exception: no specific information

Method:

How to use parameters/attributes:

- checkRushDeliveryCondition(
 - **Purpose:** to verify rush delivery conditions and mark which one is eligible.
 - **Usage:** used by PlaceOrderController to check address information provided by UI and mark which CartItem is eligible.

State

- If ‘province’ is not ‘Hanoi’, return false
- If ‘province’ is Hanoi but ‘district’ is not specified, return false.
- Else iterating through CartItem in the ‘Cart’ to mark which ‘CartItem’ is eligible.